

ARCHITECTURE REPORT

AERIS

Code Architecture

A complete walkthrough of the system structure, from configuration to output — understanding how each layer serves the mission of scalable aerospace geometry generation.

- Python
- CLI · Typer
- AeroSandbox

You do not just have "some Python files."
You now have a software system with a deliberate structure.

architecture = how you organize code, data, commands, and responsibilities so the project can grow without turning into a swamp.

INDEX

- 01

The Big Picture
- 02

Top-Level Folder Overview
- 03

Inside src/aeris/
- 04

cli.py — The Front Door
- 05

commands/ — Command Surface
- 06

common/ — Shared Infrastructure
- 07

geometry/ — Aircraft Geometry Engine
- 08

dataset/ — Batch Dataset Engine
- 09

pipeline/ — Orchestration Workflows
- 10

geometry vs dataset vs pipeline
- 11

What CLI Actually Is
- 12

Why Typer Is Used
- 13

What AeroSandbox Is Doing Here
- 14

What Manifests Are
- 15

What metadata.csv Is
- 16

Why Seeds Matter
- 17

Why "common" Code Exists
- 18

Single-Case Core + Wrappers
- 19

Why This Structure Is Standard
- 20

What Is Still Weak
- 21

The Mental Model
- 22

A Practical Analogy
- 23

Available Commands
- 24

The Journey So Far
- 25

Next Architectural Layer
- 26

Five Things to Remember
- 27

Blunt Assessment

01 The Big Picture

Your project has **four main layers**:

Configuration Layer

"What should the system do?"

Command Layer (CLI)

"How do I ask the system to do it?"

Core Logic Layer

"How is the work actually done?"

Output/Data Layer

"Where do results go, and how are they tracked?"

In your current repo, that maps roughly to:

```
configs/ → configuration layer
src/aeris/cli.py + src/aeris/commands/ → command layer
src/aeris/geometry/ , dataset/ , pipeline/ , common/ → core logic
data/ → output/data layer
```

02 Top-Level Folder Overview

configs/

Where you define inputs and behavior *without changing code*.

```
configs/geometry/wing_basic.yaml
configs/geometry/wing_bwb.yaml
configs/smoke/*.yaml
```

These YAML files are not code. They are parameter/config files that tell the program which generator family/version to use, parameter bounds, output settings, seeds, and geometry controls.

Why this matters: Without config files, every change would require editing Python code. With configs: you can rerun experiments, compare setups, keep inputs versioned, and separate "logic" from "settings."

MENTAL MODEL

Config files = "**recipes**" · Python code = "**machinery**"

data/

Where generated outputs live. Generated data should *not* be mixed with source code.

```
data/datasets/
data/debug/
data/interim/
data/processed/
data/raw/
data/runs/
```

data/runs/

Individual run outputs. Like a lab notebook entry: input config snapshot, logs, artifacts, manifest. Answers: "What happened in this one run?"

data/datasets/

Structured collections of many generated cases. A real dataset product. Answers: "What is the generated dataset as a whole?"

data/raw/ , interim/ , processed/

Standard data-engineering flow: raw → partially transformed → final cleaned data. Preparing for later ML/data workflows.

data/debug/

Temporary summaries, diagnostics, checks, inspection outputs.

dev/

Your scratch/archive area. Old prototypes, archived versions, reference code. Better than leaving dead code inside the main package like a fossil cemetery.

documents/

Project documents, reports, plans, onboarding docs. *documents/* = *human-readable planning*. *src/* = *actual software*.

scripts/

Standalone helper scripts: one-off utilities, experiments, diagnostics. Scripts are workshop tools. The CLI is the official control panel.

`src/`

The actual Python package source — the heart of the project. `src/aeris/` is the real application package.

`tests/`

Where formal tests should live. Once the project grows, "I ran it once and it seemed okay" becomes useless.

`pyproject.toml`

The project's packaging/build/dependency file. This is what lets you run `aeris dataset generate ...` instead of `python some_random_file.py`. A major step from "folder of scripts" to "software package."

03 Inside `src/aeris/` — The Actual Architecture

This is the most important part. You have these main subareas:

```
cli.py
commands/
common/
dataset/
geometry/
pipeline/
```

This is a **modular package structure**. Each area has a clear responsibility.

04 `cli.py` — The Front Door

`src/aeris/cli.py` is the main CLI entrypoint. When you type:

```
$ aeris dataset generate ...
```

that command eventually starts here. Instead of forcing the user to know which file to run, which function to call, or how imports work — the CLI gives one standard interface.

MENTAL MODEL

Think of `cli.py` as the **main receptionist**. It does not do all the work itself. It routes you to the right department.

05 `commands/` — The Official Command Surface

```
dataset.py  
geometry.py  
pipeline.py  
version.py
```

Each file defines a group of commands. Separate command files keep the root CLI clean and each command domain isolated.

MENTAL MODEL

If `cli.py` is the receptionist, `commands/` are the **department desks**.

06 `common/` — Shared Infrastructure

```
config.py – YAML config loading & helpers  
logging_utils.py – structured application logging  
paths.py – shared filesystem paths & helpers
```

These are utilities used across multiple parts of the system. Centralizing config loading, logging, and paths prevents inconsistency and bugs.

MENTAL MODEL

`common/` is the **infrastructure toolbox**: roads, pipes, electricity. Not the wing geometry itself, but the stuff the whole city depends on.

07 geometry/ — The Aircraft Geometry Engine

One of the most important modules.

```
params.py – structured parameter dataclasses
sampling.py – design point sampling
planform.py – planform geometry builder
sections.py – spanwise section geometry
aerosandbox_adapter.py – bridge to AeroSandbox
case.py – single case generation engine
export.py – artifact export
plotting.py – visualization
validation.py – geometry validation
```

params.py — Defines official shapes of inputs. Without structured models, everything becomes loose dictionaries and chaos.

case.py — One of the best architectural pieces. A reusable single geometry case generation function. Both single runs and dataset batch generation reuse this same engine. This is a classic architecture move: *extract the core unit of work, then let different workflows orchestrate it.*

aerosandbox_adapter.py — The adapter pattern. Your internal geometry model is your project's domain model. AeroSandbox is external. The adapter bridges them, reducing coupling. If you switch tools later, you don't rewrite everything.

validation.py — The system's immune system. Checks config validity, planform results, and section geometry. Without it, you don't know if outputs are meaningful or malformed nonsense.

08 dataset/ — The Batch Dataset Engine

Going from *one geometry* to *many geometries plus traceable dataset structure*.

```
dataset_run.py – main dataset generation pipeline
inspect.py – dataset QC/inspection
io.py – filesystem operations
lhs.py – Latin Hypercube Sampling
metadata.py – metadata row builder
models.py – dataset dataclasses
```

LHS gives better space-filling coverage than naive random sampling. For ML datasets, this matters a lot.

inspect.py — Dataset QC. Checks row counts, missing files, duplicates, NaNs. Generating 100 cases means nothing if 37 are broken and you don't notice.

09 pipeline/ — Orchestration Workflows

```
geometry_run.py – single-geometry run workflow
smoke.py – quick sanity checks
```

Pipelines are not the deepest core logic. They are **workflows**: take several lower-level functions and execute them in the correct order.

A smoke test is: "does the thing run at all without exploding?" Quick sanity, not deep verification.

10 The Difference Between geometry, dataset, and pipeline

geometry/

Engine Parts

How do I build one wing/BWB geometry?

dataset/

Batch / Data Product

How do I generate and manage a dataset of many geometries?

pipeline/

Workflow / Orchestration

How do I run an end-to-end process?

11 What CLI Actually Is

CLI = Command Line Interface. Instead of clicking buttons in a GUI, you type commands in the terminal.

```
$ aeris dataset generate --config ... --n 100
```

A structured way of telling the program: what action to perform, with what inputs, with what options.

Why CLI is useful: Fast. Reproducible. Scriptable. Easy to automate. Good for experiments. For your kind of platform, CLI is the right choice early on. GUI can come later.

12 Why Typer Is Used

Typer is a Python library for building CLIs cleanly. Manual CLI parsing is ugly, repetitive, and error-prone.

Typer gives: structured commands, option validation, help messages, nice syntax.

```
$ aeris dataset generate --n 100 --lhs-seed 42
```

Gets mapped cleanly to Python functions.

13 What AeroSandbox Is Doing Here

An external aerodynamic/aircraft modeling library, used as a downstream geometry representation.

IMPORTANT ARCHITECTURAL PRINCIPLE

AERIS owns the internal geometry logic. AeroSandbox is a tool AERIS can feed. If your whole project became "just AeroSandbox scripts," you would lose control of your own platform architecture.

AERIS internal geometry model first → adapter to AeroSandbox second. That is the correct hierarchy.

14 What Manifests Are

Files like `manifest.json` and `dataset_manifest.json` are run/dataset records.

They answer: what was run, with which config, when, status, counts, errors, paths.

Manifest = machine-readable audit trail. Later you will forget what produced what.

15 What metadata.csv Is

The structured table where each geometry case becomes one row — your future ML-friendly record.

Without metadata, you just have folders. Folders are okay for humans. Tables are what downstream pipelines need. `metadata.csv` is the bridge from geometry generation to data science / ML / analytics.

16 Why Seeds Matter

Generator seeds, LHS seeds, case seeds — a seed controls randomness reproducibly. Same seed = same result.

Without reproducibility: you cannot debug reliably, cannot compare experiments, your dataset becomes inconsistent.

For datasets, you did the smart thing: use LHS for design-space coverage, derive deterministic case seeds, zero out hidden perturbation randomness in dataset mode.

17 Why "common" Code Exists

A beginner mistake is copying helper code everywhere: path setup in 6 files, logger setup in 4 files, config loading in 5 files. That creates inconsistency and bugs.

The project is moving from "it works" to "it is maintainable."

18 Single-Case Core + Wrappers

One of the most important concepts in your repo.

Core

Reusable single geometry case engine

Wrapper 1

Single-run workflow

Wrapper 2

Dataset batch workflow

Define the atomic operation, build bigger workflows around it. Less duplicated code, fewer inconsistencies, easier testing, easier scaling.

19 Why Your Current Structure Is Actually Pretty Standard

Close to good software industry practice for a Python technical application. Not perfect yet, but solid.

- ✓ src/ layout for real package code
- ✓ pyproject.toml for packaging & CLI
- ✓ configs separate from code
- ✓ data separate from source
- ✓ domain modules separated by concern
- ✓ reusable common utilities
- ✓ structured CLI
- ✓ logs/manifests for reproducibility

20 What Is Still Weak / Incomplete

Tests

tests/ area needs real coverage eventually.

Documentation

Need a clean README / architecture guide.

Naming Consistency

dataset_run.py vs pipeline.py — naming can be refined.

Analysis Stage

No formal analysis/aero stage yet — next big system layer.

Schema/Versioning

Good beginnings, but needs more formal compatibility thinking.

21 The Mental Model You Should Keep

A

User-facing

CLI commands

B

Workflow

Single geometry run, dataset generation, inspection

C

Domain Logic

Sampling, planform, sections, validation, export, AeroSandbox conversion

D

Infrastructure

Config loading, logging, paths, filesystem helpers

E

Artifact/Data

Run folders, dataset folders, manifests, metadata tables, CSV/JSON/plots

If you understand these five layers, you understand the architecture.

22 A Practical Analogy

Think of AERIS like an **aircraft factory**.

configs/ → the design brief / production instructions

common/ → building utilities: power, roads, tracking

pipeline/ → production workflow for one build

data/runs/ → records of one build session

metadata.csv → the fleet inventory table

cli.py + commands/ → the control room buttons

geometry/ → manufacturing machines for one geometry

dataset/ → batch production line + warehouse records

data/datasets/ → fleet storage with catalogs

manifest.json → the build report

23 Available Commands

```
$ aeris version
```

```
$ aeris geometry ...
```

```
$ aeris dataset generate ...
```

```
$ aeris dataset inspect ...
```

```
$ aeris pipeline ...
```

One application name → domain area → action. **Clear beats clever.**

24 The Journey So Far, Architecturally

You started with "generate geometry" and matured into:

- 1 — Package structure
- 2 — CLI backbone
- 3 — Config-driven workflow
- 4 — Reusable geometry modules
- 5 — Deterministic sampling
- 6 — Explicit sample-based geometry generation
- 7 — Artifact export
- 8 — Run manifests
- 9 — Dataset orchestration
- 10 — Dataset QC

You did not jump straight to AI or optimization nonsense before building the foundation. **That was the correct move.**

25 What the Next Architectural Layer Probably Is

`analysis/` or `aero/`

Batch aerodynamic evaluation, stability metrics, label generation

`ml/`

Surrogate training and experiments

Natural roadmap: geometry creation → dataset creation → analysis labeling → ML training

26 Five Things to Remember

- 1 Separation of Responsibility**
Each module/folder should have a clear job.
- 2 CLI is the Official User Interface**
Not random scripts.
- 3 Config Files Define Behavior Without Changing Code**
Essential for reproducibility.
- 4 Core Logic Should Be Reusable**
That is why the single-case geometry engine matters.
- 5 Data Outputs Must Be Structured and Traceable**
Manifests, metadata, logs, and dataset folders matter.

27 Blunt Assessment

For someone new to software architecture, this is actually a **very respectable structure** now.

You are no longer in "random Python experimentation" territory. You have the beginnings of a **real technical platform**.

Not finished. Not polished. But **real**.

The key thing now is to understand the boundaries and not destroy them by stuffing future features into whichever file happens to be open. **That is how good architectures die.**